

BattleGrid

Software Requirements Specification

27.03.2026

Serkan EROL
150122619

Yusuf Emir ONAN
524125010

Başak AKYÜZ
150122827

Prepared for
CSE3044 Software Engineering Term Project

Table of Contents

1. INTRODUCTION	3
1.1 PURPOSE	3
1.2 SCOPE	3
1.3 DEFINITIONS, ACRONYMS, AND ABBREVIATIONS	3
1.4 REFERENCES	4
1.5 OVERVIEW	4
2. GENERAL DESCRIPTION	4
2.1 PRODUCT PERSPECTIVE	4
2.2 PRODUCT FUNCTIONS	4
2.3 USER CHARACTERISTICS	5
2.4 GENERAL CONSTRAINTS	5
2.5 ASSUMPTIONS AND DEPENDENCIES	6
3. SPECIFIC REQUIREMENTS	6
3.1 EXTERNAL INTERFACE REQUIREMENTS	6
3.1.1 <i>User Interfaces</i>	6
3.1.2 <i>Hardware Interfaces</i>	6
3.1.3 <i>Software Interfaces</i>	7
3.1.4 <i>Communications Interfaces</i>	7
3.2 FUNCTIONAL REQUIREMENTS	7
3.2.1 <i>FR1 – User Registration</i>	7
3.2.2 <i>FR2 – User Authentication</i>	7
3.2.3 <i>FR3 – Player Matchmaking</i>	8
3.2.4 <i>FR4 – Game Session Management</i>	8
3.2.5 <i>FR5 – Ship Placement Validation</i>	8
3.2.6 <i>FR6 – Turn-Based Gameplay</i>	8
3.2.7 <i>FR7 – Match Result Calculation</i>	9
3.2.8 <i>FR8 – Ranking System</i>	9
3.2.9 <i>FR9 – Leaderboard</i>	9
3.2.10 <i>FR10 – Match History</i>	9
3.2.11 <i>FR11 – Administrative Controls</i>	9
3.3 NON-FUNCTIONAL REQUIREMENTS	10
3.3.1 <i>Performance</i>	10
3.3.2 <i>Reliability</i>	10
3.3.3 <i>Availability</i>	10
3.3.4 <i>Security</i>	10
3.3.5 <i>Maintainability</i>	10
3.3.6 <i>Portability</i>	10
3.4 INVERSE REQUIREMENTS	11
3.5 DESIGN CONSTRAINTS	11
3.6 LOGICAL DATABASE REQUIREMENTS	11
3.7 OTHER REQUIREMENTS	11
4. UML DIAGRAMS	12
4.1 USE CASES	12
4.1.1 <i>Use Case #1 – Gameplay</i>	12
4.1.2 <i>Use Case #2 – Manage Account & Security</i>	13
4.1.3 <i>Use Case #3 – Review Performance</i>	13
4.2 SEQUENCE DIAGRAMS	14
A. APPENDICES	15
A.1 DECISION LOG	15

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the requirements of **BattleGrid**, a web-based multiplayer Battleship game developed as the term project for the CSE3044 Software Engineering course. The purpose of this document is to clearly describe the expected behavior, scope, and constraints of the system so that all group members share a common understanding of what the software must do.

This document is intended for the project team, the course instructor, and the teaching assistant. It will serve as the main reference for analysis, design, implementation, and testing activities throughout the project lifecycle.

1.2 Scope

BattleGrid is a web-based multiplayer version of the classic Battleship game. The system allows registered users to log in, be matched with other players of similar rank, and compete in turn based online matches. The software will provide a game environment in which players can place ships on a grid, take turns attacking their opponent's grid, and receive match results such as win or loss.

The system will also include a ranking system, a leaderboard, and match history tracking so that players can monitor their performance over time. In addition, the system will support an administrative role with authority to ban players and reset the leaderboard when necessary.

The software is intended to provide a fair, structured, and competitive multiplayer game experience. The project focuses on core game logic, user management, ranking-based matchmaking, and administrative control. Features such as an AI opponent are not included in the current scope and may be considered in future versions.

1.3 Definitions, Acronyms, and Abbreviations

SRS	Software Requirements Specification
UML	Unified Modeling Language
Player	A registered user who can join matches and play the game
Admin	A privileged user who can manage the system, including banning players and resetting the leaderboard
Matchmaking	The process of pairing two players for a match based on rank
Rank	A numerical or categorical indicator of player performance used for fair matching
Leaderboard	A ranking table that displays player standings
Match History	A record of previously played matches and their outcomes

Turn-Based Gameplay	A gameplay model in which players act one after another in a defined order
Grid	The game board on which ships are placed and attacks are made

1.4 References

- CSE3044 Software Engineering Course – SWE Project Guideline Slides.
- CSE3044 SRS Template.
- IEEE Std 830-1998 – Recommended Practice for Software Requirements Specifications.
- BattleGrid Initial Report submitted by the project group.

1.5 Overview

This document is organized into four main parts. Section 1 introduces the purpose, scope, terminology, and references of the project. Section 2 provides a general description of the system, including product perspective, major functions, user roles, constraints, and assumptions. Section 3 presents the specific functional and non-functional requirements of the system. Section 4 contains the UML diagrams prepared for the project. The appendices include supplementary material such as the decision log.

2. General Description

2.1 Product Perspective

BattleGrid is a **web-based multiplayer application** that provides an online environment for playing the classic Battleship game. The system operates as a client–server application in which players interact with the game through a web browser while the game logic and player data are managed on the server.

The system is designed as a standalone application that includes user account management, matchmaking functionality, game session management, ranking calculation, and match history tracking. All persistent data such as player accounts, rankings, match results, and administrative actions will be stored on the server.

Players interact with the system through a graphical web interface where they can register, log in, join matches, and play games. The server coordinates player matchmaking, enforces the rules of turn-based gameplay, and updates rankings and match statistics after each match.

The system also includes an administrative interface that allows administrators to manage players by banning users who violate the rules and resetting the leaderboard when necessary.

2.2 Product Functions

The BattleGrid system provides several core functions related to user management, matchmaking, gameplay, and player statistics. The main system functions are summarized below:

- User registration and authentication through a login system
- Player matchmaking based on ranking to ensure fair matches

- Creation and management of multiplayer game sessions
- Grid-based ship placement with validation rules
- Turn-based gameplay enforcing correct move order
- Match result calculation and recording of wins and losses
- Maintenance of a player ranking system and leaderboard
- Storage and display of player match history and statistics
- Administrative actions such as banning players and resetting the leaderboard

These functions together provide a structured multiplayer game environment where players can compete, track their progress, and participate in ranked matches.

2.3 User Characteristics

The system supports two main types of users: **Players** and **Administrators**.

Players

Players are the primary users of the system. They are expected to have basic familiarity with web applications and simple online games. Players interact with the system through the web interface to create accounts, log in, join matches, and play the game.

Typical player characteristics include:

- Basic computer and web browsing skills
- Ability to understand the rules of the Battleship game
- Access to a modern web browser and internet connection

Administrators

Administrators are responsible for maintaining the fairness and integrity of the system. They have additional privileges that allow them to monitor and manage player activity.

Administrator responsibilities include:

- Banning players who violate the rules
- Resetting the leaderboard when necessary
- Maintaining a fair competitive environment

Administrators are expected to have a better understanding of the system than regular players.

2.4 General Constraints

The design and implementation of the BattleGrid system are subject to several constraints that limit the possible development approaches. These constraints include:

- The system must operate as a **web-based application** accessible through standard web browsers.
- The system must support **online multiplayer gameplay**, requiring a server-based architecture capable of managing communication between players during matches.
- Player-related data such as **user accounts, rankings, leaderboard information, and match history** must be stored persistently on the server.
- The system must enforce **turn-based gameplay rules**, ensuring that players act only during their designated turns and preventing invalid actions.

2.5 Assumptions and Dependencies

The following assumptions are made during the development of the system:

- Players will have access to a stable internet connection.
- Users will access the system using modern web browsers that support standard web technologies.
- Players understand the basic rules of the Battleship game.
- The server infrastructure required to host the system will be available throughout development and testing.
- The number of simultaneous players will remain within reasonable limits for a course project.

The system may depend on standard web technologies and server infrastructure to provide multiplayer communication and data storage.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

BattleGrid provides a **web-based graphical user interface** that allows users to interact with the system through a standard web browser.

The user interface will include the following main components:

- **Authentication pages**, where users can register new accounts and log into the system.
- A **main dashboard** where players can view their ranking, access the leaderboard, and start matchmaking.
- A **game screen** where players can place ships on a grid and perform attacks on the opponent's grid.
- A **leaderboard page** displaying player rankings and statistics.
- A **match history page** showing records of previous matches and their results.

The game screen will present a **grid-based interface** where players place their ships and select grid cells to attack during their turn. The interface will visually indicate hits, misses, and remaining ships.

Administrators will have access to additional interface elements that allow them to perform administrative actions such as banning players and resetting the leaderboard.

3.1.2 Hardware Interfaces

The BattleGrid system does not require any specialized hardware components. Users interact with the system through standard computing devices such as desktop computers, laptops, or mobile devices that are capable of running modern web browsers.

The system relies only on standard input devices such as keyboards, mice, or touchscreens for user interaction.

3.1.3 Software Interfaces

The BattleGrid system interacts with server-side software components responsible for managing user accounts, matchmaking, game sessions, rankings, and match history.

The system requires a **database system** to store persistent data including:

- user account information
- player rankings
- match results
- match history
- administrative actions

These components work together to ensure that player data and gameplay state are consistently maintained during and after matches.

3.1.4 Communications Interfaces

The BattleGrid system communicates between client and server through standard **internet-based network communication**.

Player actions performed in the web interface are transmitted to the server, which processes the game logic and updates the game state accordingly. The server then sends the updated state back to the players participating in the match.

This communication mechanism enables players to interact with each other in online multiplayer matches and ensures synchronization of the game state between participants.

3.2 Functional Requirements

The functional requirements describe the core functionalities that the BattleGrid system must provide. Each requirement defines a specific behavior that the system must support in order to meet user needs and ensure correct gameplay.

3.2.1 FR1 – User Registration

Description

The system shall allow new users to create an account by providing a unique username and a password.

The system shall verify that the chosen username is not already in use.

The system shall store user account information for future authentication.

Priority: High

3.2.2 FR2 – User Authentication

Description

The system shall allow registered users to log into the system using their username and password.

The system shall verify user credentials before granting access to the system.

The system shall restrict access to gameplay and player features to authenticated users.

Priority: High

3.2.3 FR3 – Player Matchmaking

Description

The system shall allow players to request participation in a match.

The system shall match players with other players of similar ranking.

The system shall create a new game session once two suitable players are matched.

Priority: High

3.2.4 FR4 – Game Session Management

Description

The system shall create and manage a game session between two matched players.

The system shall maintain the state of the match including ship positions, attacks, and game progress.

The system shall ensure that both players receive updates about the current game state.

Priority: High

3.2.5 FR5 – Ship Placement Validation

Description

The system shall allow players to place ships on their grid before the match begins.

The system shall verify that ship placement follows the game rules.

The system shall prevent invalid ship placements such as overlapping ships or placements outside the grid.

Priority: High

3.2.6 FR6 – Turn-Based Gameplay

Description

The system shall enforce turn-based gameplay during a match.

The system shall allow only the player whose turn it is to perform an attack.

The system shall update the game state after each valid move.

Priority: High

3.2.7 FR7 – Match Result Calculation

Description

The system shall determine the outcome of a match when all ships of a player are destroyed.

The system shall record the match result as a win or loss for the participating players.

Priority: High

3.2.8 FR8 – Ranking System

Description

The system shall maintain a ranking score for each player.

The system shall update player rankings based on match outcomes.

The ranking system shall be used as a factor in the matchmaking process.

Priority: Medium

3.2.9 FR9 – Leaderboard

Description

The system shall maintain a leaderboard displaying player rankings.

The system shall allow users to view the leaderboard.

The leaderboard shall display players ordered by ranking.

Priority: Medium

3.2.10 FR10 – Match History

Description

The system shall record information about completed matches.

The system shall allow players to view their previous match results.

Match history shall include at least the opponent, match result, and match date.

Priority: Medium

3.2.11 FR11 – Administrative Controls

Description

The system shall provide administrative functions for system moderators.

Administrators shall be able to ban players from accessing the system.

Administrators shall be able to reset the leaderboard when necessary.

Priority: Medium

3.3 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints of the BattleGrid system. These requirements describe performance, reliability, availability, security, maintainability, and portability characteristics that the system must satisfy.

3.3.1 Performance

The system shall process user actions such as login, and matchmaking requests within **2 seconds under normal operating conditions**. In order the feel responsive, the system shall process user actions in game stage such as ship placements, and attack moves within **200 milliseconds under normal operating conditions**. These requirements **do not include** circumstances that are not in system's control such as network latency, and performance of user's system.

The system shall support **at least 10 concurrent active players** without significant degradation in system performance.

3.3.2 Reliability

The system shall maintain correct game state information throughout a match without inconsistencies between players.

The system shall prevent invalid game operations such as repeated attacks on the same grid cell.

3.3.3 Availability

The system shall maintain **at least 99% uptime** excluding scheduled maintenance.

System downtime shall **not exceed 10 minutes per week**.

In the event of system interruption, completed match results shall remain stored and recoverable.

3.3.4 Security

User passwords shall be stored using **secure hashing mechanisms**.

Only authenticated users shall be allowed to access gameplay features.

The system shall prevent unauthorized manipulation of match results or player rankings.

All user authentication operations shall occur through **secure communication channels**.

3.3.5 Maintainability

The system architecture shall allow developers to modify or extend system components with minimal impact on other modules.

New gameplay features or ranking adjustments shall be implementable without requiring major system redesign.

3.3.6 Portability

The system shall be accessible through **modern web browsers** without requiring platform specific installations.

The system shall support operation on major operating systems including **Windows, Linux, and macOS**.

The system shall use widely supported technologies to ensure compatibility across platforms.

3.4 Inverse Requirements

Inverse requirements describe behaviors that the system must not perform.

- The system shall not allow users to access gameplay features without authentication.
- The system shall not allow players to perform actions when it is not their turn.
- The system shall not allow invalid ship placements such as overlapping ships or placements outside the game grid.
- The system shall not allow modification of match results after a game session has ended.

3.5 Design Constraints

The system design is subject to the following constraints.

- The system shall be implemented using standard web technologies.
- The system shall support multiplayer interaction over a network connection.
- The system shall follow a client–server architecture to support player matchmaking and game session management.
- The system shall operate within typical browser limitations and network latency constraints.

3.6 Logical Database Requirements

The system shall use a database to store user and game-related information.

The database shall store the following data:

- User accounts (username and authentication credentials)
- Player ranking scores
- Match history records
- Game session data

User data shall be stored using structured data formats suitable for efficient retrieval and processing.

The system shall ensure data integrity by preventing inconsistent or corrupted records during storage and updates.

Match history and player statistics shall be retained for future reference and leaderboard calculations.

The database design shall support efficient retrieval of player information and match results.

3.7 Other Requirements

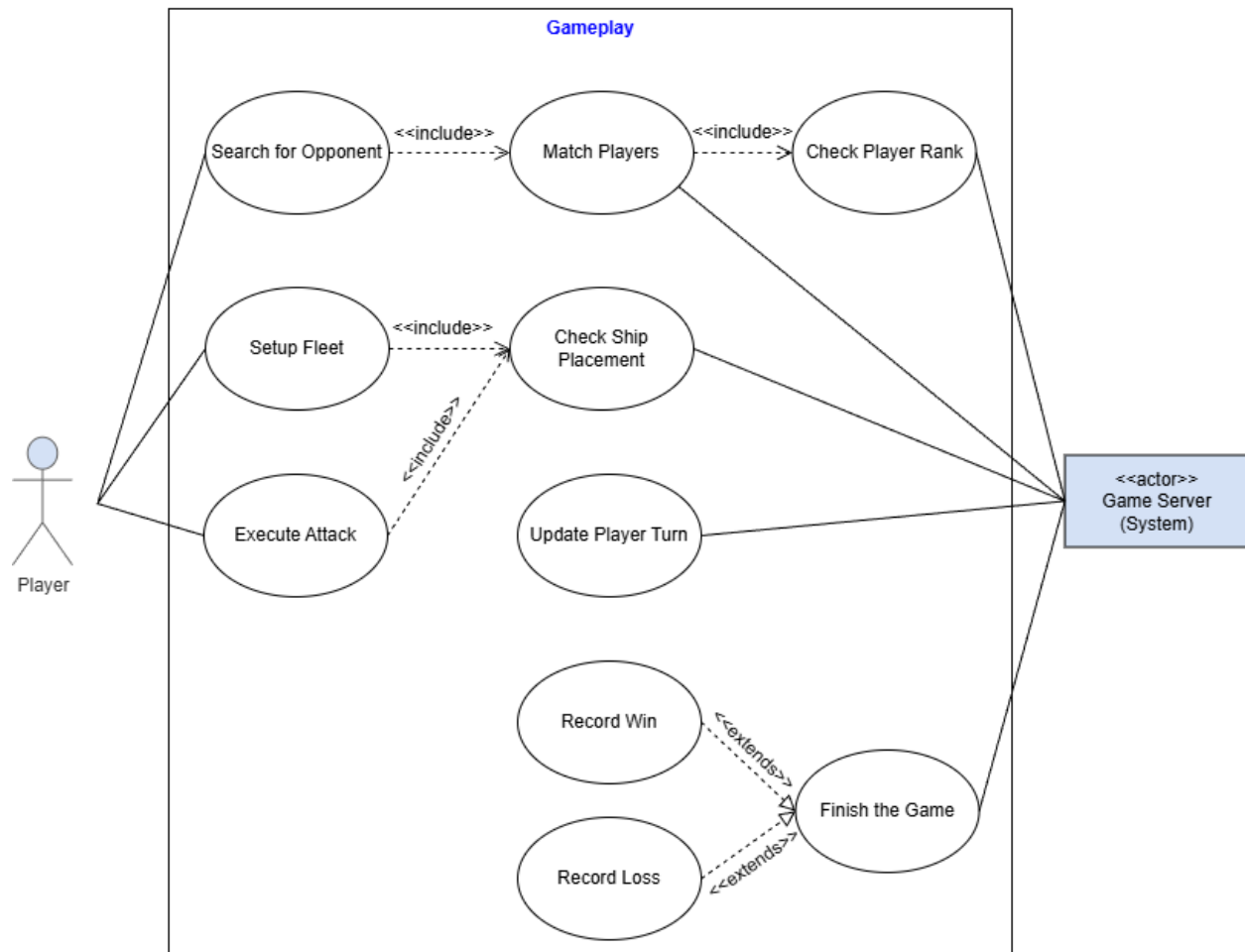
The system shall support future expansion for additional gameplay features.

The system architecture should allow integration of additional game modes or tournament systems.

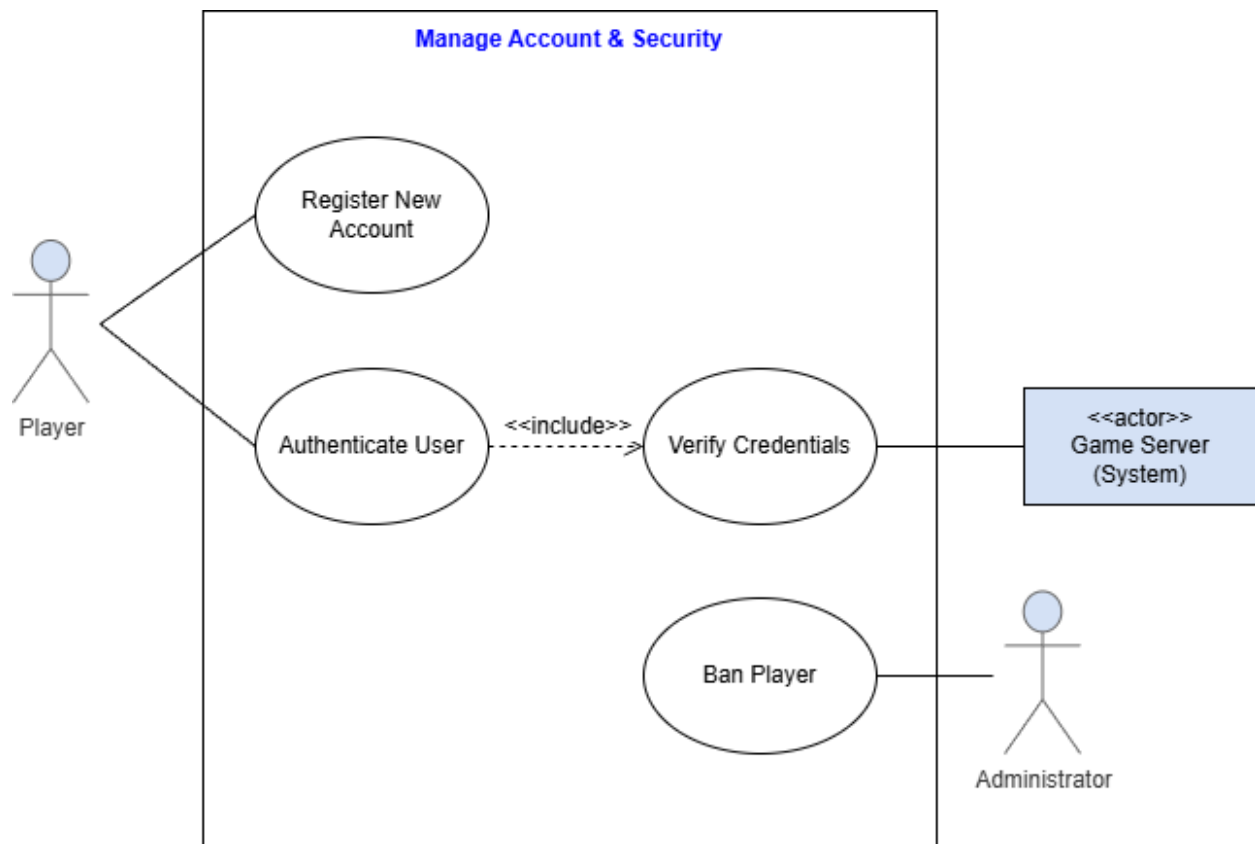
4. UML Diagrams

4.1 Use Cases

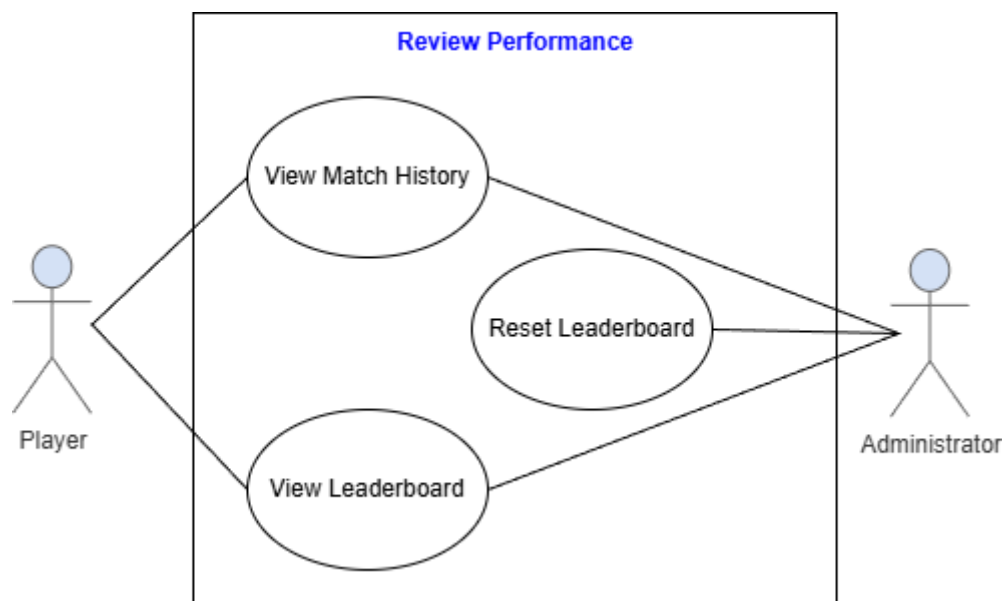
4.1.1 Use Case #1 – Gameplay



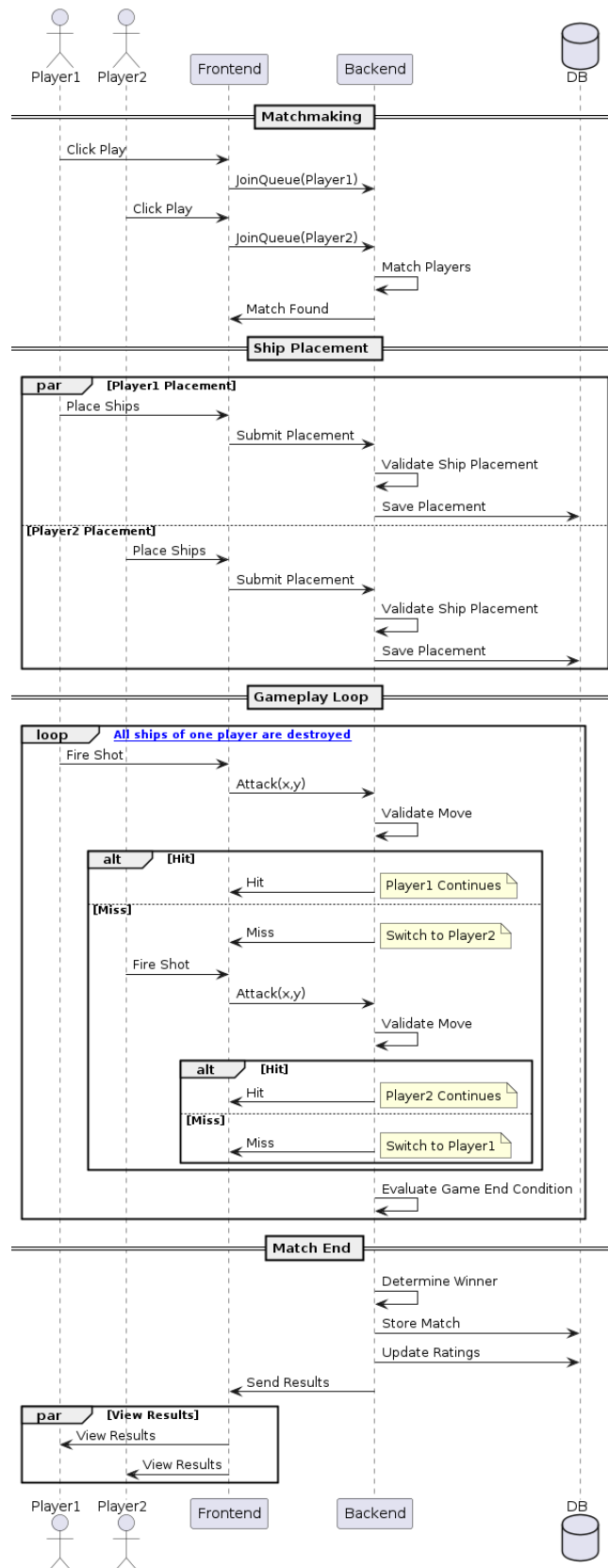
4.1.2 Use Case #2 – Manage Account & Security



4.1.3 Use Case #3 – Review Performance



4.2 Sequence Diagrams



A. Appendices

A.1 Decision Log

Matchmaking Logic

- **Decision Made:** Implement a ranking-based matchmaking system where players are paired with others of a similar score.
- **Alternatives Considered:** Random matchmaking (first in first out) or a simple level-based system.
- **Rationale:** To ensure a "fair, structured, and competitive" environment as specified in the project scope.
- **Trade-offs:** Longer wait times for players at extremely high or low ranks compared to instant random pairing.
- **Risks or Technical Debt:** If the player base is small during testing, matchmaking might fail to find a pair.

Persistent Match History

- **Decision Made:** The system will store a permanent record of every completed match, including the opponent, date, and result.
- **Alternatives Considered:** Only storing the current "Rank" score or keeping history only for the last 10 matches.
- **Rationale:** To provide long-term performance monitoring and support the "Reset Leaderboard" administrative function.
- **Trade-offs:** Increased database storage requirements over time as the number of matches grows.
- **Risks or Technical Debt:** As the MatchHistory table grows, queries for the Leaderboard or User Profile may slow down if not properly indexed.

Unique Username Constraint

- **Decision Made:** Enforce a "Unique Username" requirement during User Registration.
- **Alternatives Considered:** Allowing duplicate names with numeric tags (e.g., User#1234).
- **Rationale:** Simplifies the Leaderboard and Match History displays, ensuring players can easily identify specific opponents.
- **Trade-offs:** New users may find it difficult to find an available name as the account database grows.
- **Risks or Technical Debt:** A large number of registered but inactive accounts could eventually exhaust popular or simple usernames.

Acknowledgment: Sections of this document are based upon the IEEE Guide to Software Requirements Specification (ANSI/IEEE Std. 830-1984). The SRS templates of Dr. Orest Pilskalns (WSU, Vancouver) and Jack Hagemeister (WSU, Pullman) have also be used as guides in developing this template.